Université M'Hamed Bougara de Boumerdès (UMBB)
Faculté des Sciences – Département Informatique
Master 1 – Ingénierie des Applications (I2A / GL)

Technical Report

Waste Recycle Application

Microservices Pipeline Architecture

Module WAMS – Web & Microservices Architecture

Academic Year	2024 – 2025 | Semester 2
Project Theme	Distributed Microservices-Based System
Submitted by	Mohammed Chaib Benhebbadj Ayoub Benamrouch Mahdi Ali
Supervisor	Touazi Faycal – UMBB
Date	May 2026

Table of Contents

1. Project Context and Business Needs
2. Functional and Technical Constraints
3. System Architecture Overview
    3.1 REST API Service
    3.2 Authentication Service
    3.3 Service Discovery with Consul
    3.4 Reverse Proxy and Load Balancing with Traefik
4. Asynchronous Communication with RabbitMQ
5. Implementation Details
    5.1 Technology Stack
    5.2 API Endpoints Reference
    5.3 UI/UX Consumption
6. Deployment Strategy
    6.1 Docker and Container Orchestration
    6.3 Startup and Orchestration
7. Security Considerations
8. Conclusion

1. Project Context and Business Needs

1.1 The Waste Recycling Domain

Municipal solid waste management constitutes one of the most pressing environmental challenges facing contemporary urban societies. Inadequate sorting at the source, lack of citizen engagement, and the absence of real-time monitoring tools contribute to low recycling rates and increased landfill burden. The Waste Recycle Application addresses this gap by providing a digital platform that connects citizens with local recycling infrastructure and empowers municipal administrators with actionable data analytics.

The application operates within a smart-city paradigm, leveraging artificial intelligence (AI) for waste classification, a recommendation engine for citizen guidance, and an event-driven notification layer to ensure timely communication between all stakeholders.

1.2 Target Users

User Role	Responsibilities
Citizens	Register, submit waste images for AI classification, receive recycling tips, and locate the nearest collection point.

1.3 Problem Statement and Value Proposition

Existing waste management portals are monolithic, difficult to scale during peak usage (e.g., collection days), and tightly couple business logic with UI rendering. This project proposes a fully decoupled microservices architecture that:

- Isolates each concern (authentication, AI inference, recommendations, notifications) into independently deployable services.
- Enables horizontal scaling of high-load services (e.g., the AI service) without affecting others.
- Provides asynchronous processing for time-consuming tasks, preventing API timeouts.
- Implements modern service-discovery mechanisms to avoid hard-coded inter-service URLs.

2. Functional and Technical Constraints

2.1 Functional Requirements

The system must expose a complete CRUD interface across all domain entities.

2.2 Security Constraints

- All API routes (except /api/auth/register and /api/auth/login) must require a valid JWT Bearer token.
- Passwords must be stored using Django's PBKDF2 hashing algorithm (never in plaintext).
- Role-based access control (RBAC) must distinguish Admin privileges from standard User access.
- JWT tokens must carry a user UUID, email, and role claim; expiry must not exceed 24 hours.
- All inter-service communication must occur within a private Docker network (backend bridge).

2.3 Multi-Server Deployment Constraint

The assignment explicitly mandates that each logical component runs on a distinct server instance. The following mapping satisfies this constraint:

Instance	Role	Description
Instance 1	Frontend	React / Flutter mobile client (handled by partner)
Instance 2	Auth	Django Auth Service – JWT issuance & validation
Instance 3	API	Django AI Service – YOLO inference + Cloudinary storage
Instance 4	API	Django Recommendation Service – recycling tips engine
Instance 5	Gateway	Django API Gateway – routing, middleware, Consul lookup
Instance 6	Worker	Notification Service – RabbitMQ consumer + SMTP mailer
Instance 7	Broker	RabbitMQ – AMQP message broker
Instance 8	Registry	Consul – service registration and health checks
Instance 9	Proxy	Traefik – reverse proxy, load balancer, dashboard
Instance 10	Storage	MySQL (one isolated instance per service)

3. System Architecture Overview

The system follows a Domain-Driven Microservices pattern. Each service owns its data store, exposes a versioned REST API, and communicates through either synchronous HTTP (for real-time queries routed via the Gateway) or asynchronous AMQP messages (for background tasks routed via RabbitMQ).

Figure 1 – High-level System Architecture

3.1 REST API Service

The core business logic of the application is split across three independent Django REST Framework services:

AI Service

Responsible for all AI inference. Accepts multipart image uploads, runs the custom-trained YOLOv8 model (best.pt) to detect and classify waste objects, uploads both the original and annotated result images to Cloudinary CDN, and persists a Submission record.

Recommendation Service

Stores and retrieves recycling guidance records keyed by waste type. Consumed by the Gateway after AI inference to enrich the response with actionable tips.

3.2 Authentication Service

Built on Django's AbstractBaseUser with a UUID primary key and email as the login field. JWT tokens are issued via djangorestframework-simplejwt and validated stateless in the Gateway without any database round-trip.

User Model

Field	Description
id	UUID (primary key, auto-generated)
email	Unique – used as USERNAME_FIELD
username	CharField(max_length=50)
password	Hashed via PBKDF2 (set_password)
is_active	BooleanField – account activation
is_staff	BooleanField – admin privileges
created_at	DateTimeField(auto_now_add=True)

Token Flow

1. POST /api/auth/register/ – creates user, hashes password, returns 201.
2. POST /api/auth/login/ – validates credentials, returns access + refresh JWT pair.
3. POST /api/auth/refresh/ – exchanges refresh token for a new access token.
4. GET /api/auth/me/ – returns authenticated user profile (requires Bearer token).

Gateway JWT Middleware

The Gateway service implements a custom Django middleware that intercepts every inbound request, decodes the JWT using the shared SIGNING_KEY, and injects the user_id

into a custom request header (X-User-Id) forwarded to downstream services. Downstream services never query the Auth database; they simply read the trusted header.

3.3 Service Discovery with Consul

HashiCorp Consul is deployed as a single-node development agent and serves as the central service registry for the entire platform.

Each Django service self-registers on startup via the AppConfig.ready() hook using the python-consul client library.

A retry loop (up to 10 attempts, 3-second back-off) is wrapped around the registration call to tolerate race conditions during container startup ordering.

Service Discovery in the Gateway

The Gateway uses Consul's /v1/agent/services REST endpoint to resolve a service name to a live address:port at request time. A lightweight in-memory cache reduces Consul round-trips for frequently-called services.

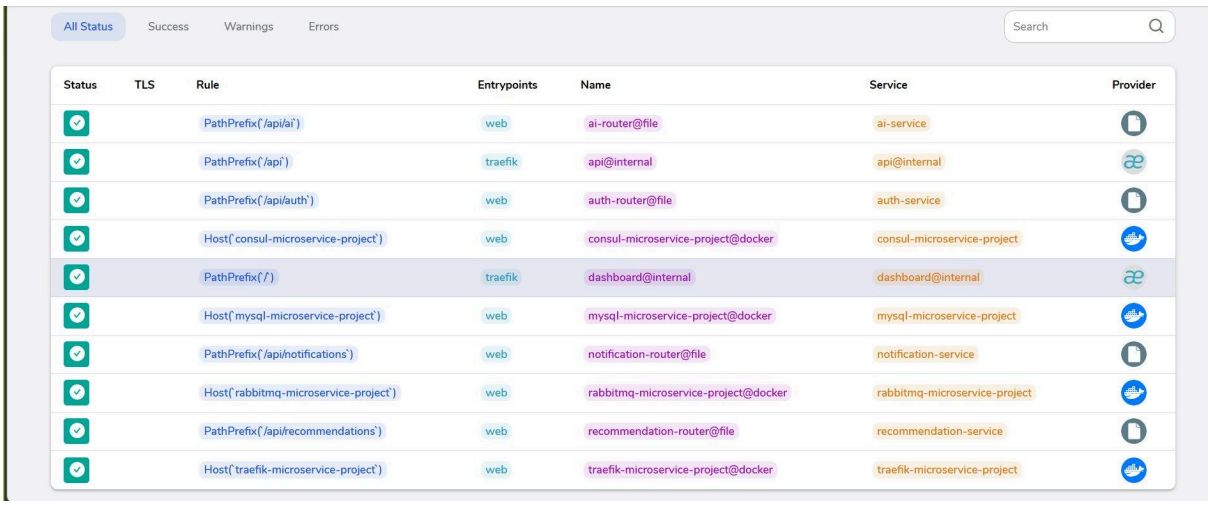
Parameter	Value
Consul UI	http://localhost:8500
Health Check Interval	10 seconds (configurable via .env)
Registered Services	auth-service, ai-service, recommendation-service, gateway-service

3.4 Reverse Proxy and Load Balancing with Traefik

Traefik v3.0 acts as the single external entry point for all client traffic. It dynamically discovers routing rules through two providers simultaneously:

- Docker provider – reads labels attached to each container in the Compose network.
- Consul Catalog provider – polls Consul's service catalog for additional routing metadata, enabling future service-mesh scenarios.

The Traefik dashboard is accessible at http://localhost:8080 for real-time routing visualisation and health monitoring.



4. Asynchronous Communication with RabbitMQ

RabbitMQ 3.13 (with the Management Plugin) serves as the AMQP message broker for all background and event-driven workflows. Its use decouples producer services from consumer services, prevents blocking API responses during heavy operations, and provides built-in message durability and retry semantics.

4.1 Primary Async Flow – Registration Confirmation Email

The most critical asynchronous flow is triggered upon successful user registration.

The consumer is launched as a daemon thread inside the Django NotificationConfig.ready() hook, ensuring it starts automatically with the service without blocking the WSGI server startup.

4.5 RabbitMQ Configuration

Parameter	Value
Broker Image	rabbitmq:3-management
AMQP Port	5672 (application connections)
Management UI	http://localhost:15672 (guest / guest)
Queue Name	notifications (durable=True)
Event Types	USER_REGISTERED
Connection	python-pika client library

5. Implementation Details

5.1 Technology Stack

Component	Technology
Backend Framework	Django 5.x + Django REST Framework
AI Inference	YOLOv8 (Ultralytics) – custom trained model (best.pt)
Image Storage	Cloudinary CDN (Python SDK)
Authentication	djangorestframework-simplejwt (HS256)
Message Broker	RabbitMQ 3.13 + python-pika
Service Registry	HashiCorp Consul)
Reverse Proxy	Traefik v3.0
Database	MySQL 8 (one isolated instance per service)
Containerisation	Docker + Docker Compose
Email	SMTP via Gmail App Password
Frontend	React / Flutter (handled by partner)

5.2 Key API Endpoints – Gateway (port 8004)

Method	Endpoint	Description
POST	/api/auth/register/	Register a new user account
POST	/api/auth/login/	Authenticate and receive JWT pair
POST	/api/auth/refresh/	Exchange refresh token for new access token
GET	/api/auth/me/	Retrieve authenticated user profile
POST	/api/gateway/upload/	Upload waste image for AI inference
GET	/api/recommendations/{waste_type}/	Fetch recycling tips for a waste type

5.3 UI/UX Consumption Pattern

The frontend application (mobile or web) interacts exclusively with the API Gateway on port 8004 (or via Traefik at api.local). It never addresses individual microservices directly.

A typical citizen interaction sequence:

5. User opens app → POST /api/auth/login/ → receives JWT access token.
6. App stores token in secure local storage.
7. User captures waste photo → POST /api/ai/upload/ with Bearer token + multipart image.
8. Gateway validates JWT (middleware), extracts user UUID, forwards to AI Service with X-User-Id header.
9. AI Service runs YOLO inference, uploads images to Cloudinary, saves Submission record, returns JSON.
10. App displays annotated image URL + waste_type + confidence from JSON response.

11. App calls GET /api/recommendations/{waste_type}/ to display recycling tips.
12. Background: RabbitMQ delivers USER_REGISTERED email notification asynchronously.

6. Deployment Strategy

6.1 Docker Compose Orchestration

For local development and demonstration, all server instances are simulated as Docker containers within a shared bridge network named backend. Services communicate using Docker's internal DNS (container name as hostname), eliminating hard-coded IP addresses.

6.2 Startup and Orchestration Script

startup sequence:

13. Detect Docker Compose version
14. Start infrastructure containers: Consul, RabbitMQ, MySQL instances.
15. Wait for database health checks to pass.
16. Execute makemigrations + migrate for each Django service.
17. Launch each Django service
18. Report endpoint URLs for immediate access.

7. Security Considerations

The architecture incorporates multiple security layers aligned with OWASP API Security Top-10 recommendations:

Security Layer	Implementation
JWT Stateless Auth	Access tokens signed with HS256; expiry enforced; no server-side session state.
UUID Primary Keys	User IDs are UUIDs, preventing enumeration attacks on sequential integer IDs.
Password Hashing	Django PBKDF2-SHA256 with 600,000 iterations; never stored in plaintext.
Network Isolation	All inter-service traffic stays within the Docker backend bridge network; only Traefik exposes port 80 externally.
Header Injection Guard	Gateway validates JWT before injecting X-User-Id; downstream services reject requests missing this header.
No Shared DB	Services cannot access each other's databases; data leakage surface is minimised.
Secrets via .env	No credentials committed to source control; .env listed in .gitignore.
App Password	SMTP Gmail authentication uses Application Passwords (not account password), compatible with 2FA.

8. Conclusion

This project demonstrates a production-grade microservices pipeline applied to the waste management domain. By decomposing the monolithic concern into independent services (Auth, AI, Recommendation, Gateway, Notification), the architecture achieves:

- Independent deployability and horizontal scalability per service.
- Asynchronous, non-blocking processing for time-consuming tasks.
- Resilient service discovery via Consul with automatic health checks.
- Secure, stateless authentication with JWT and UUID-based identity.
- Dynamic routing and load balancing via Traefik.